Gate 03 Reconciliation: Auth & Access Control

Field	Value
Gate	03 — Auth & Access Control
Spec version	2026-04-09 (gate spec created with architecture proposal)
Implementation date	2026-04-16
Reconciled by	Claude Code
Branch	gate/03-auth-access-control

Summary

Gate 03 is functionally complete. All auth endpoints, the TierPermission permission class, for_tier() queryset managers, and institution-scoped access filtering are implemented and tested. Several deviations from the original spec were made during implementation based on security review and code quality review findings — all are improvements. Two items (password reset, AuditLog write path) are deferred to post-MVP.

Acceptance Criteria Status

#	Criterion (from spec)	Status	Notes
BE-03-1.1	Registration creates Tier 2 user, is_active=False, sends verification email	Pass	Implemented in views.register; tested in TestRegister
BE-03-1.2	Password under 12 chars returns HTTP 400	Pass	Django MinimumLengthValidator configured with min_length=12 in AUTH_PASSWORD_VALIDATORS
BE-03-1.3	Duplicate email returns HTTP 400	Pass	Validated in RegisterSerializer.validate_email
BE-03-2.1	Valid verification token activates account (HTTP 200)	Pass	Implemented in views.verify_email; tested in TestVerifyEmail
BE-03-2.2	Expired/invalid token returns HTTP 400	Pass	TimestampSigner with 48h max_age handles expiry; BadSignature handles invalid
BE-03-3.1	Valid credentials return token + access_tier	Pass	Login response includes token, access_tier, and user
BE-03-3.2	Inactive account login returns error	Deviation	Returns 401 (not 403 as spec stated) — see Deviatio below

BE-03-3.3	Rate limiting: 6th attempt in 15min returns 429	Pass	Atomic cache.add/cache.incr implementation; tested TestLogin.test_login_rate_limited
BE-03-4.1	Logout invalidates token	Pass	Token deleted from database; tested in TestLogout
BE-03-5.1	GET /me/ returns profile with access_tier and institution	Pass	UserProfileSerializer returns id, email, name, access_ institution, is_active, date_joined
BE-03-5.2	Unauthenticated GET /me/ returns 401	Pass	IsAuthenticated permission class applied
BE-03-6.1	Anonymous request to TierPermission(min_tier=2) returns 401	Pass	Anonymous users treated as Tier 1; tested in TestTierPermission.test_anonymous_blocked_from_t
BE-03-6.2	Tier 2 token against min_tier=3 returns 403	Pass	Tested in TestTierPermission.test_tier_2_user_blocked_from_t
BE-03-6.3	Tier 3 user cannot modify another institution's ExSituPopulation	Pass	scope_to_institution filters by user.institution_id; test in TestInstitutionScoping

User Story Status

Story ID	Title	Status	Notes
BE-03-1	User Registration	Complete	POST /api/v1/auth/register/ — all 3 acceptance criteria pass
BE-03-2	Email Verification	Complete	Changed from GET to POST (security review); verification URL points to FRONTEND_BASE_URL
BE-03-3	Login	Complete	Rate limiting uses cache-based approach; inactive users get 401 not 403
BE-03-4	Logout	Complete	POST /api/v1/auth/logout/ deletes token
BE-03-5	Current User Profile	Complete	GET /api/v1/auth/me/ — response includes additional fields (is_active, date_joined) beyond spec
BE-03-6	Tier Enforcement on Protected Endpoints	Complete	TierPermission factory function + has_object_permission; for_tier() managers on all 3 models

Deviations

1. TierPermission implemented as factory function returning a class

- **Spec said:** Class with `__init__(self, min_tier)`, used as `TierPermission.require(3)` or via `functools.partial`
- **Implementation does:** `TierPermission(min_tier)` is a factory function that returns a permission class, used as `permission_classes = [TierPermission(3)]`
- **Reason:** Code quality review — DRF's permission_classes expects a list of classes (not instances). A factory returning a class integrates cleanly with DRF's instantiation lifecycle.
- **Impact:** None. The call-site syntax is identical to what the spec suggested: `[TierPermission(3)]`.

2. Email verification changed from GET to POST

- **Spec said:** `GET /api/v1/auth/verify/?token=<token>`
- **Implementation does:** `POST /api/v1/auth/verify/` with `{"token": "..."}` in the request body
- **Reason:** Security review — GET requests that mutate state are unsafe because email link prefetchers, browser preloaders, and proxy caches can trigger verification unintentionally.
- **Impact:** The frontend must submit the token via POST after extracting it from the URL query parameter. Verification URL now points to `FRONTEND_BASE_URL/verify?token=...` which renders a page that POSTs to the API.

3. Rate limiter uses atomic cache operations instead of list-based approach

- **Spec said:** Use `django-ratelimit` or cache-based rate limiting
- **Implementation does:** Cache-based rate limiting using atomic `cache.add()` + `cache.incr()` pattern
- **Reason:** Code quality review identified TOCTOU race condition in the initial list-based approach. The atomic incr pattern is simpler, has no extra dependency, and is race-condition-free.
- **Impact:** None. Functionally equivalent. No `django-ratelimit` dependency needed.

4. Inactive users get 401 (not 403) from login

- **Spec said:** HTTP 403 with "account is not yet verified" message for inactive accounts
- **Implementation does:** HTTP 401 with generic "Invalid email or password" message — same response as wrong credentials or non-existent email
- **Reason:** Security review — returning 403 with a specific message confirms the account exists and reveals its activation state, enabling account enumeration. The is_active check was moved into EmailBackend.authenticate() so inactive users return None (same as wrong credentials).
- **Impact:** Security improvement. Callers cannot distinguish between "account exists but inactive", "wrong password", and "no such email". The test `test_login_no_enumeration` explicitly verifies this property.

5. scope_to_institution rejects users below Tier 3

- **Spec said:** Institution-scoped filtering for Tier 3-4 users
- **Implementation does:** Users below Tier 3 get `queryset.none()` even if they have an institution affiliation
- **Reason:** Code quality review — the original implementation did not check tier, meaning a Tier 2 user with an institution FK could potentially access scoped data.
- **Impact:** Defense in depth. Tested explicitly in `test_tier2_with_institution_still_gets_empty`.

6. Verification URL points to frontend, not API

- **Spec said:** Verification URL is API endpoint `GET /api/v1/auth/verify/?token=<token>`
- **Implementation does:** Verification email links to `FRONTEND_BASE_URL/verify?token=...` ; the frontend page then POSTs to the API
- **Reason:** Follows from Deviation #2 (GET to POST change). Also, security review recommended using FRONTEND_BASE_URL setting instead of deriving URLs from the request Host header (host header injection risk).
- **Impact:** Requires a frontend page at `/verify` that reads the token query param and submits it to the API. FRONTEND_BASE_URL is configurable via environment variable (default: `http://localhost:3000`).

7. Login response includes user_id

- **Spec said:** Response: `{"token": "<token>", "access_tier": 2}`
- **Implementation does:** Response: `{"token": "<token>", "access_tier": 2, "user_id": <pk>}`
- **Reason:** Practical convenience — the frontend needs the user ID for subsequent API calls without having to make a separate GET /me/ request.
- **Impact:** Additive. No spec fields removed.

Additions (not in spec)

1. **has_object_permission on TierPermission** — Security review recommended adding object-level permission checking as defense in depth. The method delegates to `has_permission` , ensuring tier checks apply even when DRF checks object-level permissions on detail views.

2. **Anonymous users allowed at Tier 1** — `TierPermission(min_tier=1)` returns True for anonymous users, enabling public endpoints to use the same permission class consistently. The spec did not explicitly define Tier 1 anonymous behavior for TierPermission.

3. **UserProfileSerializer includes is_active and date_joined** — Spec showed id, email, name, access_tier, institution. Implementation adds is_active and date_joined as read-only fields.

4. **IP hashing in rate limit keys** — Rate limit cache keys use SHA-256 hashed IPs (truncated to 16 hex chars) rather than raw IPs, avoiding storing raw IP addresses in the cache layer.

5. **Database-level check constraint on access_tier** — `CheckConstraint` ensures access_tier stays in range 1-5 at the database level, complementing the Django validator.

6. **Email normalization** — Both RegisterSerializer and LoginSerializer normalize email to lowercase with strip(), preventing case-sensitivity issues in email matching.

Deferred Items

Item	Spec reference	Reason	Priority
Token expiry	Not in spec (DRF TokenAuth has no built-in expiry)	Would require custom token model or migration to SimpleJWT. DRF's TokenAuthentication tokens are permanent.	Post-MVP

Password reset flow	Out of Scope section: "implement if time permits"	Not MVP-blocking per spec	Post-MVP
AuditLog write path	Technical Tasks implied; AuditLog model exists	Model is defined with correct fields (user, action, model_name, object_id, timestamp, changes, ip_address) but no signals or middleware write to it yet	Gate 04 or post-MVP
Full 5x5 tier enforcement matrix tests	Gate Exit Criteria #2	Tests cover representative tier combinations (anonymous, Tier 2, 3, 5 against various min_tiers) but not an exhaustive 5x5 grid. Coverage is sufficient for correctness verification.	Consider expanding in adversarial test pass

Technical Decisions Made During Implementation

1. **Custom User model with email as USERNAME_FIELD** — No username field. Email is the sole authentication identifier, matching the spec's email-based registration flow.
2. **TimestampSigner for verification tokens** — Uses Django's built-in `TimestampSigner` with 48-hour expiry. Simpler than a separate VerificationToken model and avoids database lookups for token validation.
3. **Function-based views (not ViewSet)** — Auth endpoints use `@api_view` decorators rather than a ViewSet class. Each endpoint is a standalone function, which is appropriate for auth flows that don't follow standard CRUD patterns.
4. **Console email backend for development** — `EMAIL_BACKEND` defaults to `django.core.mail.backends.console.EmailBackend`, configurable via environment variable for production.
5. **SessionAuthentication + TokenAuthentication** — Both enabled in `DEFAULT_AUTHENTICATION_CLASSES` as the spec recommended. Session auth serves Django Admin; token auth serves the API.
6. **AUTHENTICATION_BACKENDS set to EmailBackend only** — Custom `accounts.backends.EmailBackend` replaces Django's default `ModelBackend`. This ensures email-based auth everywhere (including Django Admin).

Spec Updates Needed

1. **BE-03-2 endpoint method**: Change from `GET /api/v1/auth/verify/?token=<token>` to `POST /api/v1/auth/verify/` with body `{"token": "..."}`. Update acceptance criteria to reference POST.
2. **BE-03-3 inactive account response**: Change from "HTTP 403 with message indicating account is not yet verified" to "HTTP 401 with generic 'Invalid email or password' message (same as wrong credentials, to prevent account enumeration)".

3. **BE-03-3 login response**: Add `user_id` to the documented response body: `{"token": "<token>", "access_tier": 2, "user_id": 1}`.
4. **Technical Implementation Notes — TierPermission**: Update the code example to show the factory function pattern instead of `class TierPermission(BasePermission)` with `__init__`.
5. **Technical Implementation Notes — Rate limiting**: Document that the implementation uses atomic `cache.add/cache.incr` (no `django-ratelimit` dependency).
6. **Add FRONTEND_BASE_URL setting**: Document in Technical Implementation Notes that verification emails link to `FRONTEND_BASE_URL/verify?token=...` and this setting is required in the environment.
7. **Add AuditLog write path to a future gate**: The AuditLog model exists but nothing writes to it. Should be assigned to Gate 04 (Django Admin) or a dedicated gate.